

## **4.10 분류 실습 — 신용카드 사기 검출**

## 목차

---

1. 실습 개요와 데이터 소개
  1. 문제 정의
  2. 컬럼 구성
  3. 데이터 로딩
2. 데이터 일차 가공 및 모델 학습/예측/평가
  1. 전처리 함수와 데이터 분리 함수
  2. 레이블 분포 확인
  3. 평가 함수 준비
  4. 로지스틱 회귀 성능
  5. 모델 학습·평가 통합 함수
  6. LightGBM 성능
3. 데이터 분포도 변환 후 재평가
  1. Amount 분포 확인
  2. StandardScaler 적용
  3. 로그 변환 적용
4. 이상치 데이터 제거 후 재평가
  1. 상관관계 히트맵
  2. IQR 기반 이상치 검출 함수
  3. V14 이상치 제거와 성능
5. SMOTE 오버 샘플링 적용
  1. SMOTE 개념과 적용
  2. 로지스틱 회귀 결과
  3. 정밀도-재현율 곡선 분석
  4. LightGBM 결과
6. 핵심 요약 및 머신러닝 활용 포인트

## 1. 실습 개요와 데이터 소개

### 1.1 문제 정의

2013년 9월 유럽 카드 사용자의 신용카드 거래 데이터로, 거래가 사기(fraud)인지 정상인지를 판별하는 이진 분류 문제입니다. 전체 284,807건 중 사기는 492건(약 0.172%)에 불과한, 4.9장보다도 훨씬 극심한 불균형 데이터입니다.

#### ■ 이 실습에서 배우는 것

이번 장은 같은 알고리즘에 전처리 방법만 바꿔 가며 성능 변화를 관찰하는 구조입니다.

1. 기본 — Time 컬럼만 제거
2. StandardScaler — Amount 표준화
3. 로그 변환 — Amount의 왜곡된 분포 보정
4. 이상치 제거 — IQR 기반으로 극단값 삭제
5. SMOTE — 소수 클래스 오버 샘플링

각 단계마다 로지스틱 회귀와 LightGBM을 함께 평가해, 전처리가 알고리즘별로 다르게 작용한다는 점을 확인합니다.

### 1.2 컬럼 구성

| 컬럼       | 설명                                                           |
|----------|--------------------------------------------------------------|
| Time     | 첫 거래로부터 경과한 초. 예측에 무의미하므로 제거합니다.                             |
| V1 ~ V28 | PCA로 변환된 익명 피쳐. 개인정보 보호를 위해 원본 의미가 감춰져 있습니다. 이미 스케일링된 상태입니다. |
| Amount   | 거래 금액. 유일하게 PCA 변환되지 않은 피쳐라 스케일이 다릅니다.                       |
| Class    | 레이블. 0 = 정상, 1 = 사기                                          |

### 1.3 데이터 로딩

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
%matplotlib inline

card_df = pd.read_csv('./creditcard.csv')
card_df.head(3)
```

| 줄 | 코드                                        | 상세 설명                                    |
|---|-------------------------------------------|------------------------------------------|
| 8 | card_df = pd.read_csv('./creditcard.csv') | 약 150MB의 CSV를 로딩합니다. 284,807건 × 31컬럼입니다. |

#### ▶ 실행 결과 (앞 8개 컬럼만 표시)

|   | Time | V1        | V2        | V3       | V4       | V5        | V6        | V7        |
|---|------|-----------|-----------|----------|----------|-----------|-----------|-----------|
| 0 | 0.0  | -1.359807 | -0.072781 | 2.536347 | 1.378155 | -0.338321 | 0.462388  | 0.239599  |
| 1 | 0.0  | 1.191857  | 0.266151  | 0.166480 | 0.448154 | 0.060018  | -0.082361 | -0.078803 |
| 2 | 1.0  | -1.358354 | -1.340163 | 1.773209 | 0.379780 | -0.503198 | 1.800499  | 0.791461  |

```
shape: (284807, 31)
Class
0    284315
1     492
Name: count, dtype: int64
```

### ⚠ 극단적 불균형 — 0.172%

사기 거래가 492건 / 284,807건 = 0.172%입니다. "모두 정상"이라고만 답해도 **정확도 99.83%**가 나옵니다. 3.1장에서 배운 **정확도의 함정**이 극단적으로 나타나는 사례로, **재현율과 정밀도, F1, AUC**를 함께 봐야만 합니다.

## 2. 데이터 일차 가공 및 모델 학습/예측/평가

### 2.1 전처리 함수와 데이터 분리 함수

```
from sklearn.model_selection import train_test_split

# 인자로 입력받은 DataFrame을 복사 한 뒤 Time 컬럼만 삭제하고 복사된 DataFrame 반환
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    df_copy.drop('Time', axis=1, inplace=True)
    return df_copy
```

| 줄 | 코드                                         | 상세 설명                                                                            |
|---|--------------------------------------------|----------------------------------------------------------------------------------|
| 4 | def get_preprocessed_df(df=None):          | <b>이 실습의 핵심 설계</b> 입니다. 전처리 로직을 함수로 분리해 두고, <b>이 함수만 바꿔 가며</b> 여러 전처리 방식을 실험합니다. |
| 5 | df_copy = df.copy()                        | <b>반드시 복사</b> 합니다. 원본을 직접 수정하면 다음 실험에서 <b>이미 변형된 데이터</b> 를 쓰게 되어 결과가 오염됩니다.      |
| 6 | df_copy.drop('Time', axis=1, inplace=True) | Time은 <b>단순 경과 시간</b> 이라 사기 여부와 무관합니다. 복사본에 대한 inplace이므로 안전합니다.                 |

```
# 사전 데이터 가공 후 학습과 테스트 데이터 세트를 반환하는 함수.
def get_train_test_dataset(df=None):
    df_copy = get_preprocessed_df(df)
    # DataFrame의 맨 마지막 컬럼이 레이블, 나머지는 피쳐들
    X_features = df_copy.iloc[:, :-1]
    y_target = df_copy.iloc[:, -1]
    # train_test_split( )으로 학습과 테스트 데이터 분할. stratify=y_target으로 Stratified 기반 분할
    X_train, X_test, y_train, y_test = \
        train_test_split(X_features, y_target, test_size=0.3, random_state=0, stratify=y_target)
    return X_train, X_test, y_train, y_test

X_train, X_test, y_train, y_test = get_train_test_dataset(card_df)
```

| 줄   | 코드                                | 상세 설명                                                                        |
|-----|-----------------------------------|------------------------------------------------------------------------------|
| 3   | df_copy = get_preprocessed_df(df) | 전처리 함수를 <b>내부에서 호출</b> 합니다. 덕분에 전처리 함수만 바꾸면 <b>이 함수는 수정 없이</b> 새 전처리가 적용됩니다. |
| 5~6 | X_features / y_target             | 맨 마지막 컬럼(Class)을 레이블로 분리합니다.                                                 |

|   |                                                                      |                                                                                                                       |
|---|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 9 | <code>train_test_split(..., test_size=0.3, stratify=y_target)</code> | <b>stratify=y_target</b> 이 결정적입니다. 사기가 0.172%뿐이라 무작위 분할하면 테스트 세트에 사기가 거의 없거나 편중될 수 있습니다. 계층적 분할로 양쪽 모두 0.172%를 유지합니다. |
|---|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|

## 2.2 레이블 분포 확인

```
print('학습 데이터 레이블 값 비율')
print(y_train.value_counts()/y_train.shape[0] * 100)
print('테스트 데이터 레이블 값 비율')
print(y_test.value_counts()/y_test.shape[0] * 100)
```

### ▶ 실행 결과

```
학습 데이터 레이블 값 비율
Class
0    99.827451
1     0.172549
Name: count, dtype: float64
테스트 데이터 레이블 값 비율
Class
0    99.826785
1     0.173215
Name: count, dtype: float64
```

### ■ 결과 해석

학습 0.1725% / 테스트 0.1732%로 소수점 셋째 자리까지 거의 일치합니다. `stratify` 가 제대로 작동한 것입니다. 4.9장에서 `stratify` 없이 0.27%p 차이가 났던 것과 대조됩니다.

## 2.3 평가 함수 준비

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import roc_auc_score

def get_clf_eval(y_test, pred=None, pred_proba=None):
    confusion = confusion_matrix(y_test, pred)
    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
    f1 = f1_score(y_test, pred)
    roc_auc = roc_auc_score(y_test, pred_proba)
    print('오차 행렬')
    print(confusion)
    print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f}, \
F1: {3:.4f}, AUC: {4:.4f}'.format(accuracy, precision, recall, f1, roc_auc))
```

| 줄  | 코드                                                                 | 상세 설명                                            |
|----|--------------------------------------------------------------------|--------------------------------------------------|
| 4  | <code>def get_clf_eval(y_test, pred=None, pred_proba=None):</code> | 3.5장에서 완성한 평가 함수를 그대로 재사용합니다. 5개 지표를 한 번에 확인합니다. |
| 10 | <code>roc_auc_score(y_test, pred_proba)</code>                     | AUC만 예측 확률을 받습니다.                                |

## 2.4 로지스틱 회귀 성능

```
from sklearn.linear_model import LogisticRegression

lr_clf = LogisticRegression()
lr_clf.fit(X_train, y_train)
lr_pred = lr_clf.predict(X_test)
lr_pred_proba = lr_clf.predict_proba(X_test)[: , 1]

# 3장에서 사용한 get_clf_eval() 함수를 이용하여 평가 수행.
get_clf_eval(y_test, lr_pred, lr_pred_proba)
```

### ▶ 실행 결과

```
오차 행렬
[[85281  14]
 [  57  91]]
정확도: 0.9992, 정밀도: 0.8667, 재현율: 0.6149,   F1: 0.7194, AUC:0.9741
(학습 14.8초)
```

### ■ 오차 행렬 해석

|           | 예측 0 (정상)   | 예측 1 (사기) |
|-----------|-------------|-----------|
| 실제 0 (정상) | TN = 85,281 | FP = 14   |
| 실제 1 (사기) | FN = 57     | TP = 91   |

정확도는 99.92%지만 재현율은 61.49%에 불과합니다. 실제 사기 148건 중 57건을 놓쳤습니다. 사기 검출에서 FN은 실제 금전 피해로 직결되므로 가장 치명적인 오류입니다.

정확도가 99.92%나 되는 이유는 정상 거래가 99.83%이기 때문입니다. 정확도만 보면 완벽해 보이지만 실제로는 사기의 40%를 놓치고 있는 모델입니다.

## 2.5 모델 학습·평가 통합 함수

```
# 인자로 사이킷런의 Estimator 객체와, 학습/테스트 데이터 세트를 입력 받아서 학습/예측/평가 수행.
def get_model_train_eval(model, ftr_train=None, ftr_test=None, tgt_train=None, tgt_test=None):
    model.fit(ftr_train, tgt_train)
    pred = model.predict(ftr_test)
    pred_proba = model.predict_proba(ftr_test)[: , 1]
    get_clf_eval(tgt_test, pred, pred_proba)
```

| 줄   | 코드                                                    | 상세 설명                                                                                  |
|-----|-------------------------------------------------------|----------------------------------------------------------------------------------------|
| 2   | def get_model_train_eval(model, ftr_train=None, ...): | 학습 → 예측 → 평가를 하나로 묶은 함수입니다. <code>model</code> 을 인자로 받으므로 어떤 알고리즘이든 같은 코드로 처리할 수 있습니다. |
| 3~5 | fit / predict / predict_proba                         | 사이킷런 Estimator 인터페이스가 통일되어 있어 가능한 설계입니다(2.3장).                                         |
| 6   | get_clf_eval(tgt_test, pred, pred_proba)              | 앞의 평가 함수를 호출합니다. 함수 안에서 함수를 부르는 계층 구조로, 이후 실험이 한 줄 호출로 끝납니다.                           |

## 2.6 LightGBM 성능

```
from lightgbm import LGBMClassifier

lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1, boost_from_average=False)
get_model_train_eval(lgbm_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train, tgt_test=y_test)
```

| 줄 | 코드                                                                 | 상세 설명                                                                                                                                                                               |
|---|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3 | <code>LGBMClassifier(n_estimators=1000, num_leaves=64, ...)</code> | 트리 1000개, 리프 64개로 <b>충분히 복잡한 모델</b> 을 만듭니다. 데이터가 28만 건으로 크기 때문입니다.                                                                                                                  |
| 3 | <code>boost_from_average=False</code>                              | <b>매우 중요한 파라미터입니다.</b> LightGBM은 기본적으로 <b>레이블 평균에서 부스팅을 시작</b> 하는데, 이 데이터처럼 <b>극심한 불균형</b> 에서는 시작점이 0에 지나치게 가까워져 <b>성능이 크게 저하</b> 됩니다. <code>False</code> 로 두면 0에서 시작해 안정적으로 학습됩니다. |
| 4 | <code>get_model_train_eval(lgbm_clf, ...)</code>                   | 앞서 만든 통합 함수 덕분에 <b>한 줄로 학습·예측·평가</b> 가 끝납니다.                                                                                                                                        |

### ▶ 실행 결과

```
오차 행렬
[[85290   5]
 [  36 112]]
정확도: 0.9995, 정밀도: 0.9573, 재현율: 0.7568,   F1: 0.8453, AUC:0.9790
(학습 16.5초)
```

### ■ 결과 해석 — LightGBM의 우위

| 지표        | 로지스틱 회귀 | LightGBM      |
|-----------|---------|---------------|
| 정밀도       | 0.8667  | <b>0.9573</b> |
| 재현율       | 0.6149  | <b>0.7568</b> |
| F1        | 0.7194  | <b>0.8453</b> |
| AUC       | 0.9741  | <b>0.9790</b> |
| 놓친 사기(FN) | 57건     | <b>36건</b>    |

모든 지표에서 LightGBM이 우수합니다. 특히 **재현율이 0.6149 → 0.7568**로 크게 향상되어, 놓치는 사기가 **57건에서 36건으로 21건 줄었습니다.** 트리 기반 모델이 **비선형 패턴**을 포착하는 능력이 선형 모델보다 뛰어나기 때문입니다.

## 3. 데이터 분포도 변환 후 재평가

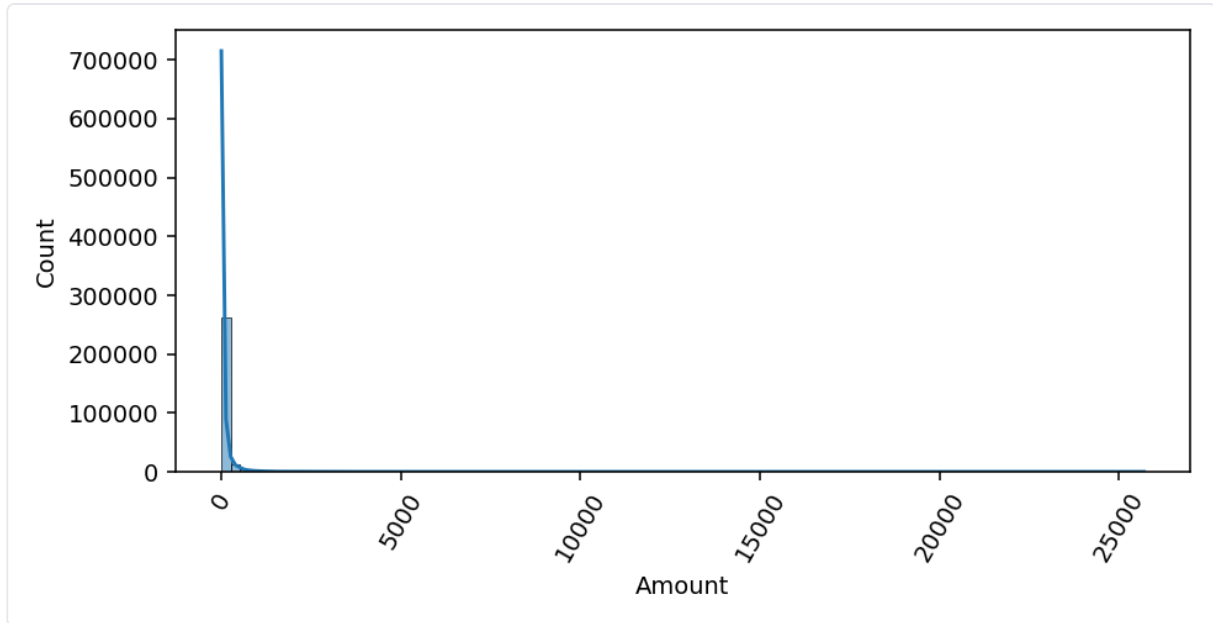
### 3.1 Amount 분포 확인

```
import seaborn as sns

plt.figure(figsize=(8, 4))
plt.xticks(range(0, 30000, 1000), rotation=60)
sns.histplot(card_df['Amount'], bins=100, kde=True)
plt.show()
```

| 줄 | 코드                                                               | 상세 설명                                                                                   |
|---|------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| 4 | <code>plt.xticks(range(0, 30000, 1000), rotation=60)</code>      | x축 눈금을 <b>1000 간격</b> 으로 촘촘히 표시하고 <code>rotation=60</code> 으로 <b>60도 기울여</b> 겹침을 방지합니다. |
| 5 | <code>sns.histplot(card_df['Amount'], bins=100, kde=True)</code> | 히스토그램을 그립니다. <code>kde=True</code> 는 <b>커널 밀도 추정 곡선</b> 을 함께 그려 분포 형태를 부드럽게 보여 줍니다.     |

#### ▶ 실행 결과



거래 금액(Amount) 분포 — 극단적인 오른쪽 꼬리

#### ■ 결과 해석

대부분의 거래가 **0~1,000** 사이에 극도로 몰려 있고, 오른쪽으로 **매우 긴 꼬리**가 이어집니다. 최댓값은 25,691입니다. 이런 분포를 **right-skewed(오른쪽으로 치우친)** 분포라고 합니다.

로지스틱 회귀 같은 선형 모델은 **데이터가 정규 분포에 가까울 때** 성능이 좋으므로, 이 왜곡을 보정하면 성능이 개선될 가능성이 있습니다.

## 3.2 StandardScaler 적용

```
from sklearn.preprocessing import StandardScaler
# 사이킷런의 StandardScaler를 이용하여 정규분포 형태로 Amount 피쳐값 변환하는 로직으로 수정.
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    scaler = StandardScaler()
    amount_n = scaler.fit_transform(df_copy['Amount'].values.reshape(-1, 1))
    # 변환된 Amount를 Amount_Scaled로 피쳐명 변경후 DataFrame맨 앞 컬럼으로 입력
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    # 기존 Time, Amount 피쳐 삭제
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    return df_copy
```

| 줄 | 코드 | 상세 설명 |
|---|----|-------|
|---|----|-------|

|    |                                                               |                                                                                                                                                                                     |
|----|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3  | def get_preprocessed_df(df=None):                             | 같은 이름으로 함수를 재정의합니다. 파이썬은 나중 정의가 이전 것을 덮어쓰므로, <code>get_train_test_dataset()</code> 은 자동으로 새 전처리를 사용하게 됩니다. 이것이 이 실습 구조의 핵심입니다.                                                      |
| 6  | scaler.fit_transform(df_copy['Amount'].values.reshape(-1, 1)) | <ul style="list-style-type: none"> <li><code>.values</code> : Series → ndarray</li> <li><code>reshape(-1, 1)</code> : 스케일러는 2차원만 받으므로 (n, 1)로 변환 결과는 평균 0, 분산 1로 표준화됩니다.</li> </ul> |
| 8  | df_copy.insert(0, 'Amount_Scaled', amount_n)                  | <code>insert(위치, 컬럼명, 값)</code> 으로 맨 앞(0번 위치)에 새 컬럼을 삽입합니다. 단순 대입과 달리 위치를 지정할 수 있습니다.                                                                                               |
| 10 | df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)        | 원본 Amount와 Time을 삭제합니다. 변환된 컬럼만 남깁니다.                                                                                                                                               |

## ▶ 실행 결과

```
### 로지스틱 회귀 예측 성능 ###
오차 행렬
[[85281  14]
 [  55  93]]
정확도: 0.9992, 정밀도: 0.8692, 재현율: 0.6284,   F1: 0.7294, AUC:0.9706

### LightGBM 예측 성능 ###
오차 행렬
[[85290   5]
 [  37 111]]
정확도: 0.9995, 정밀도: 0.9569, 재현율: 0.7500,   F1: 0.8409, AUC:0.9779
```

## ■ 결과 해석 — 미미한 변화

| 모델       | 전처리            | 재현율           | F1            | AUC           |
|----------|----------------|---------------|---------------|---------------|
| 로지스틱     | 기본             | 0.6149        | 0.7194        | 0.9741        |
|          | StandardScaler | <b>0.6284</b> | <b>0.7294</b> | 0.9706        |
| LightGBM | 기본             | <b>0.7568</b> | <b>0.8453</b> | <b>0.9790</b> |
|          | StandardScaler | 0.7500        | 0.8409        | 0.9779        |

- 로지스틱 회귀는 재현율이 소폭 향상(0.6149 → 0.6284)되었습니다. 선형 모델이라 스케일링에 반응합니다.
- LightGBM은 오히려 미세하게 하락했습니다. 트리 모델은 값의 크기가 아니라 분할 기준만 쓰므로 스케일링이 본질적으로 무의미하며, 차이는 무작위 변동 수준입니다.

2.5장에서 배운 "트리 계열은 스케일링이 불필요하다"는 원칙이 실증적으로 확인됩니다.

## 3.3 로그 변환 적용

```
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    # 넘파이의 log1p()를 이용하여 Amount를 로그 변환
    amount_n = np.log1p(df_copy['Amount'])
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    return df_copy
```

출 코드

상세 설명

|   |                                        |                                                                                                                                                                                                                       |
|---|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4 | amount_n = np.log1p(df_copy['Amount']) | <b>log1p(x) = log(1+x)</b> 입니다. 그냥 <b>log(x)</b> 를 쓰지 않는 이유는 <b>Amount가 0인 거래가 존재하는데 log(0)은 -무한대</b> 가 되기 때문입니다. 1을 더하면 최소값이 <b>log(1)=0</b> 이 되어 안전합니다.<br>로그 변환은 <b>큰 값을 압축</b> 해 오른쪽 꼬리를 줄이고 분포를 정규 분포에 가깝게 만듭니다. |
|---|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## ▶ 실행 결과

```

### 로지스틱 회귀 예측 성능 ###
오차 행렬
[[85282  13]
 [  59  89]]
정확도: 0.9992, 정밀도: 0.8725, 재현율: 0.6014,   F1: 0.7120, AUC:0.9734

### LightGBM 예측 성능 ###
오차 행렬
[[85290   5]
 [  35 113]]
정확도: 0.9995, 정밀도: 0.9576, 재현율: 0.7635,   F1: 0.8496, AUC:0.9796

```

## ■ 결과 해석 — 알고리즘별로 정반대 반응

| 모델          | 기본     | Scaler        | 로그 변환           |
|-------------|--------|---------------|-----------------|
| 로지스틱 F1     | 0.7194 | <b>0.7294</b> | 0.7120 ↓        |
| LightGBM F1 | 0.8453 | 0.8409        | <b>0.8496</b> ↑ |

로지스틱 회귀는 오히려 나빠졌고, LightGBM은 좋아졌습니다. 전처리가 모든 모델에 일률적으로 좋은 것이 아니라는 중요한 교훈입니다.

LightGBM 기준으로는 **로그 변환이 가장 우수**하므로, 이후 단계에서는 로그 변환을 유지합니다.

## 4. 이상치 데이터 제거 후 재평가

### 4.1 상관관계 히트맵

```

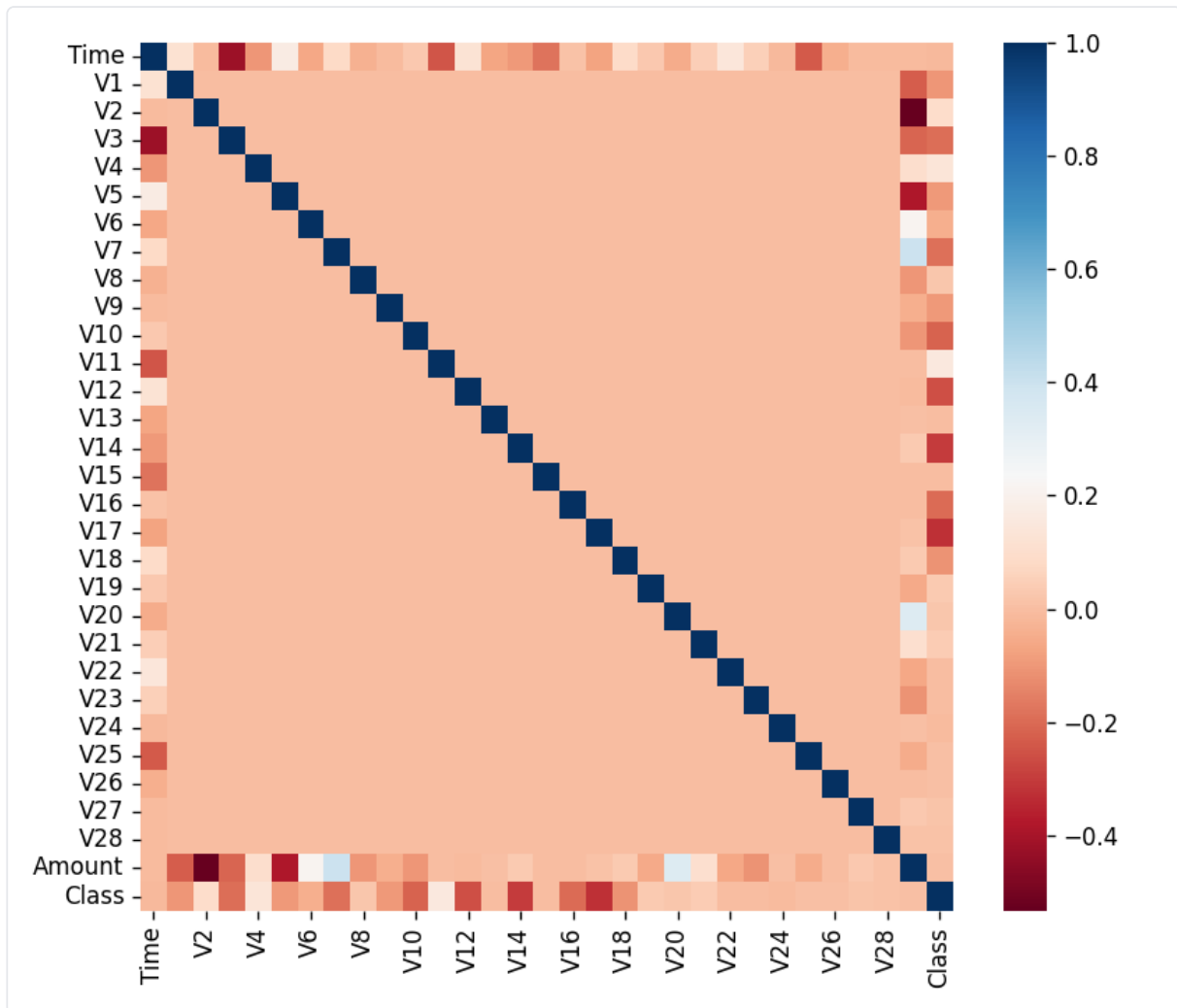
import seaborn as sns

plt.figure(figsize=(9, 9))
corr = card_df.corr()
sns.heatmap(corr, cmap='RdBu')

```

| 줄 | 코드                             | 상세 설명                                                                                                                  |
|---|--------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 4 | corr = card_df.corr()          | 모든 컬럼 쌍의 <b>피어슨 상관계수</b> 를 계산해 31×31 행렬을 만듭니다. -1(완전 음의 상관) ~ 1(완전 양의 상관) 범위입니다.                                       |
| 5 | sns.heatmap(corr, cmap='RdBu') | 상관 행렬을 <b>색상</b> 으로 표현합니다. <b>'RdBu'</b> 는 <b>빨강(음의 상관) ~ 파랑(양의 상관)</b> 의 발산형 컬러맵으로, <b>0을 기준으로 양방향</b> 을 구분하기에 적합합니다. |

## ▶ 실행 결과



전체 피처의 상관관계 히트맵

#### ■ 결과 해석 — 왜 V14를 골랐는가

V1~V28은 PCA로 변환된 피처라 서로 거의 무상관(대각선만 진하고 나머지는 옅음)입니다. PCA의 정의상 당연한 결과입니다.

주목할 것은 맨 아래 Class 행입니다. V14와 V17이 Class와 가장 강한 음의 상관(진한 빨강)을 보입니다. 즉 사기 여부와 가장 밀접한 피처라는 뜻이며, 다음 단계에서 V14를 이상치 제거 대상으로 선택하는 근거가 됩니다.

레이블과 상관이 높은 피처의 이상치를 제거하는 것이 가장 효과가 크기 때문입니다.

## 4.2 IQR 기반 이상치 검출 함수

```
import numpy as np

def get_outlier(df=None, column=None, weight=1.5):
    # fraud에 해당하는 column 데이터만 추출, 1/4 분위와 3/4 분위 지점을 np.percentile로 구함.
    fraud = df[df['Class']==1][column]
    quantile_25 = np.percentile(fraud.values, 25)
    quantile_75 = np.percentile(fraud.values, 75)
    # IQR을 구하고, IQR에 1.5를 곱하여 최대값과 최소값 지점 구함.
    iqr = quantile_75 - quantile_25
    iqr_weight = iqr * weight
    lowest_val = quantile_25 - iqr_weight
    highest_val = quantile_75 + iqr_weight
```

```
# 최대값 보다 크거나, 최소값 보다 작은 값을 아웃라이어로 설정하고 DataFrame index 반환.
outlier_index = fraud[(fraud < lowest_val) | (fraud > highest_val)].index
return outlier_index
```

| 줄     | 코드                                                                                                        | 상세 설명                                                                                                                         |
|-------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 5     | <code>fraud = df[df['Class']==1][column]</code>                                                           | <b>매우 중요한 설계입니다. 사기 데이터(Class=1)만</b> 대상으로 이상치를 찾습니다. 정상 거래까지 포함하면 <b>사기의 특성 자체가 이상치로</b> 잡혀 버리기 때문입니다.                       |
| 6~7   | <code>np.percentile(fraud.values, 25) / 75</code>                                                         | <b>1사분위수(Q1)와 3사분위수(Q3)</b> 를 구합니다. 각각 하위 25%, 75% 지점의 값입니다.                                                                  |
| 9     | <code>iqr = quantile_75 - quantile_25</code>                                                              | <b>IQR(사분위 범위) = Q3 - Q1.</b> 데이터 중앙 50%가 퍼져 있는 폭입니다.                                                                         |
| 10    | <code>iqr_weight = iqr * weight</code>                                                                    | IQR에 <b>1.5를 곱합니다.</b> 이 1.5는 <b>통계학의 관례적 기준</b> 으로, 정규 분포에서 약 99.3%를 포함하는 범위에 해당합니다.                                         |
| 11~12 | <code>lowest_val = quantile_25 - iqr_weight</code><br><code>highest_val = quantile_75 + iqr_weight</code> | 이상치 판정 <b>하한과 상한</b> 입니다. 이 범위를 벗어나면 이상치로 봅니다.                                                                                |
| 14    | <code>fraud[(fraud &lt; lowest_val)   (fraud &gt; highest_val)].index</code>                              | 불린 인덱싱으로 범위를 벗어난 행을 찾고, <b>값이 아니라 인덱스</b> 를 반환합니다. 나중에 <code>drop( )</code> 으로 삭제하기 위함입니다. <code> </code> 는 <b>OR</b> 연산자입니다. |

```
outlier_index = get_outlier(df=card_df, column='V14', weight=1.5)
print('이상치 데이터 인덱스:', outlier_index)
```

#### ▶ 실행 결과

```
이상치 데이터 인덱스: Index([8296, 8615, 9035, 9252], dtype='int64')
```

#### ■ 결과 해석

사기 거래 492건 중 **단 4건**이 V14의 이상치로 검출되었습니다. 전체 28만 건에서 4건을 지우는 **극히 미세한 조작**입니다. 다음 절에서 이 4건이 성능에 얼마나 영향을 주는지 확인합니다.

## 4.3 V14 이상치 제거와 성능

```
# get_processed_df( )를 로그 변환 후 V14 피처의 이상치 데이터를 삭제하는 로직으로 변경.
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    amount_n = np.log1p(df_copy['Amount'])
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    # 이상치 데이터 삭제하는 로직 추가
    outlier_index = get_outlier(df=df_copy, column='V14', weight=1.5)
    df_copy.drop(outlier_index, axis=0, inplace=True)
    return df_copy
```

| 줄   | 코드                                                                             | 상세 설명                                                  |
|-----|--------------------------------------------------------------------------------|--------------------------------------------------------|
| 4~6 | 로그 변환 부분                                                                       | 앞 절의 <b>로그 변환을 유지</b> 합니다. LightGBM 기준으로 가장 좋았기 때문입니다. |
| 8   | <code>outlier_index = get_outlier(df=df_copy, column='V14', weight=1.5)</code> | 이상치 인덱스를 구합니다.                                         |

|   |                                                   |                                                                        |
|---|---------------------------------------------------|------------------------------------------------------------------------|
| 9 | df_copy.drop(outlier_index, axis=0, inplace=True) | <b>axis=0</b> 이므로 <b>행(로우)</b> 을 삭제합니다. 컬럼 삭제(axis=1)와 혼동하지 않도록 주의하세요. |
|---|---------------------------------------------------|------------------------------------------------------------------------|

#### ▶ 실행 결과

```

### 로지스틱 회귀 예측 성능 ###
오차 행렬
[[85280  15]
 [  48  98]]
정확도: 0.9993, 정밀도: 0.8673, 재현율: 0.6712,   F1: 0.7568, AUC:0.9725

### LightGBM 예측 성능 ###
오차 행렬
[[85290   5]
 [  25 121]]
정확도: 0.9996, 정밀도: 0.9603, 재현율: 0.8288,   F1: 0.8897, AUC:0.9780

```

#### ■ 결과 해석 — 4건 삭제의 놀라운 효과

| 모델       | 단계       | 재현율           | F1            | 놓친 사기      |
|----------|----------|---------------|---------------|------------|
| 로지스틱     | 로그 변환    | 0.6014        | 0.7120        | 59건        |
|          | + 이상치 제거 | <b>0.6712</b> | <b>0.7568</b> | <b>48건</b> |
| LightGBM | 로그 변환    | 0.7635        | 0.8496        | 35건        |
|          | + 이상치 제거 | <b>0.8288</b> | <b>0.8897</b> | <b>25건</b> |

단 4건을 삭제했을 뿐인데 두 모델 모두 큰 폭으로 향상되었습니다.

- 로지스틱 재현율 **+7.0%p**, LightGBM 재현율 **+6.5%p**
- LightGBM이 놓치는 사기가 **35건 → 25건**으로 10건 줄었습니다.

극단적인 이상치 몇 개가 **모델의 결정 경계를 왜곡**하고 있었던 것입니다. **데이터 품질 개선이 모델 튜닝보다 효과적일 수 있다**는 것을 보여 주는 강력한 사례입니다.

## 5. SMOTE 오버 샘플링 적용

### 5.1 SMOTE 개념과 적용

SMOTE(Synthetic Minority Over-sampling Technique)는 적은 데이터 세트에 있는 개별 데이터들의 K 최근접 이웃을 찾아, 그 사이에 새로운 데이터를 합성해 소수 클래스를 증식시키는 기법입니다. 단순 복제가 아니라 **새로운 가상 데이터를 만들어 낸다**는 점이 핵심입니다.

```

#conda install -c conda-forge imbalanced-learn

from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=0)
X_train_over, y_train_over = smote.fit_resample(X_train, y_train)
print('SMOTE 적용 전 학습용 피쳐/레이블 데이터 세트: ', X_train.shape, y_train.shape)
print('SMOTE 적용 후 학습용 피쳐/레이블 데이터 세트: ', X_train_over.shape, y_train_over.shape)
print('SMOTE 적용 후 레이블 값 분포: \n', pd.Series(y_train_over).value_counts())

```

| 줄 | 코드                                                    | 상세 설명                                                                                                                                                                             |
|---|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3 | <code>from imblearn.over_sampling import SMOTE</code> | <code>imbalanced-learn</code> 은 사이킷런과 별개의 패키지입니다. <code>pip install imbalanced-learn</code> 으로 설치합니다.                                                                             |
| 6 | <code>smote.fit_resample(X_train, y_train)</code>     | 반드시 학습 데이터에만 적용합니다. 테스트 데이터에 SMOTE를 쓰면 절대 안 됩니다 — 존재하지 않는 가짜 데이터로 평가하는 셈이 되기 때문입니다. 메서드 이름이 <code>fit_transform</code> 이 아니라 <code>fit_resample</code> 인 이유는 행의 개수 자체가 바뀌기 때문입니다. |

#### ▶ 실행 결과

```
SMOTE 적용 전 학습용 피쳐/레이블 데이터 세트: (199362, 29) (199362,)
SMOTE 적용 후 학습용 피쳐/레이블 데이터 세트: (398040, 29) (398040,)
SMOTE 적용 후 레이블 값 분포:
Class
0    199020
1    199020
Name: count, dtype: int64
```

#### ■ 결과 해석

학습 데이터가 199,362건 → 398,040건으로 약 2배 증가했습니다. 사기 데이터가 342건에서 199,020건으로 약 582배 증식되어 정확히 1:1 균형이 되었습니다.

## 5.2 로지스틱 회귀 결과

```
lr_clf = LogisticRegression()
# ftr_train과 tgt_train 인자값이 SMOTE 증식된 X_train_over와 y_train_over로 변경됨에 유의
get_model_train_eval(lr_clf, ftr_train=X_train_over, ftr_test=X_test, tgt_train=y_train_over, tgt_test=y_test)
```

| 줄 | 코드                                                                          | 상세 설명                                                                                |
|---|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 3 | <code>ftr_train=X_train_over,</code><br><code>tgt_train=y_train_over</code> | 학습 데이터만 SMOTE 적용본으로 바꿉니다. <code>ftr_test</code> 와 <code>tgt_test</code> 는 원본 그대로입니다. |

#### ▶ 실행 결과

```
오차 행렬
[[82933 2362]
 [ 11 135]]
정확도: 0.9722, 정밀도: 0.0541, 재현율: 0.9247, F1: 0.1022, AUC:0.9736
```

### △ 결과 해석 — 재현율은 폭등, 정밀도는 붕괴

| 지표     | SMOTE 전 | SMOTE 후       | 변화           |
|--------|---------|---------------|--------------|
| 재현율    | 0.6712  | <b>0.9247</b> | <b>+0.25</b> |
| 정밀도    | 0.8673  | <b>0.0541</b> | <b>-0.81</b> |
| F1     | 0.7568  | 0.1022        | <b>-0.65</b> |
| FP(오탐) | 15건     | <b>2,362건</b> | 157배         |

재현율은 **92.47%**로 급등했지만 정밀도가 **5.41%**로 붕괴했습니다. 사기라고 예측한 2,497건 중 실제 사기는 135건뿐입니다.

원인은 명확합니다. SMOTE로 사기 데이터를 582배 증식하자 모델이 "사기가 절반이나 되는 세상"을 학습했습니다. 그러나 실제 테스트 데이터에서 사기는 **0.17%**뿐이므로, 모델이 사기를 지나치게 남발하게 된 것입니다.

실무 관점에서 정상 거래 2,362건을 사기로 차단하면 고객 불만이 폭증합니다. 사용할 수 없는 모델입니다.

## 5.3 정밀도-재현율 곡선 분석

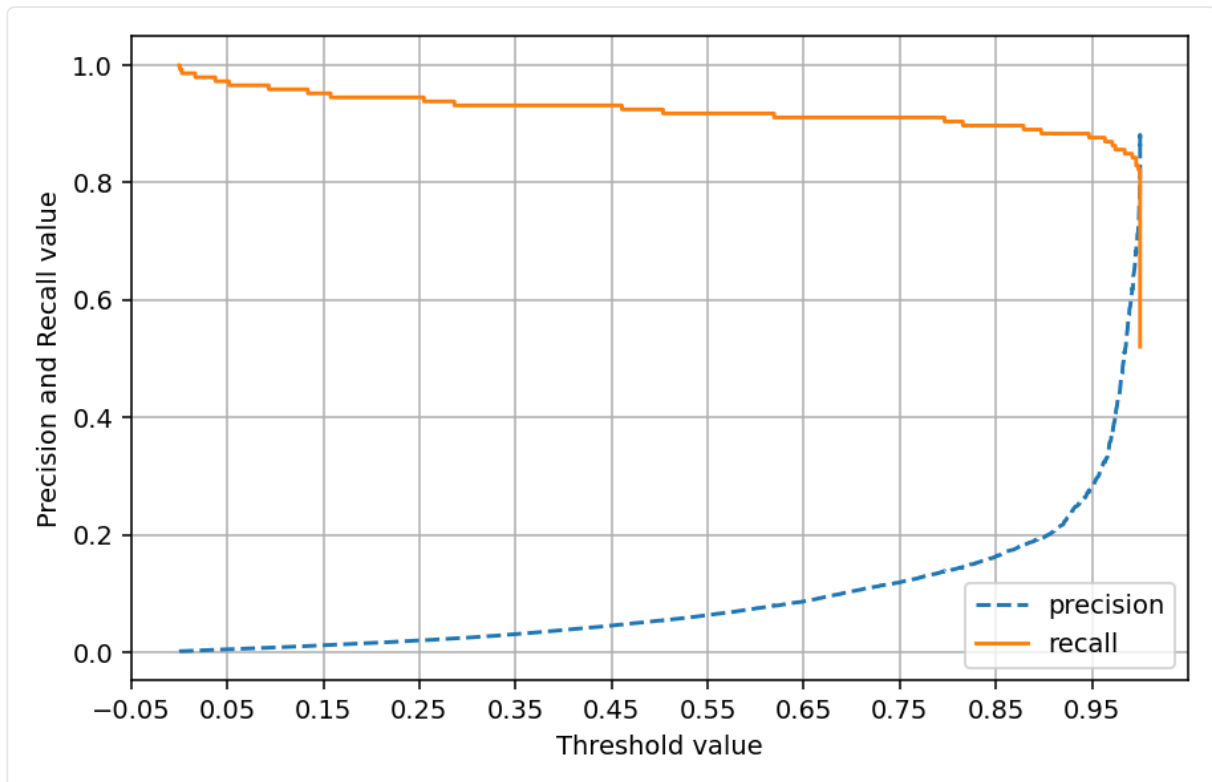
```
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from sklearn.metrics import precision_recall_curve
%matplotlib inline

def precision_recall_curve_plot(y_test, pred_proba_c1):
    precisions, recalls, thresholds = precision_recall_curve(y_test, pred_proba_c1)
    plt.figure(figsize=(8,6))
    threshold_boundary = thresholds.shape[0]
    plt.plot(thresholds, precisions[0:threshold_boundary], linestyle='--', label='precision')
    plt.plot(thresholds, recalls[0:threshold_boundary], label='recall')
    start, end = plt.xlim()
    plt.xticks(np.round(np.arange(start, end, 0.1), 2))
    plt.xlabel('Threshold value'); plt.ylabel('Precision and Recall value')
    plt.legend(); plt.grid()
    plt.show()

precision_recall_curve_plot(y_test, lr_clf.predict_proba(X_test)[: , 1])
```

| 줄  | 코드                                                                       | 상세 설명                                             |
|----|--------------------------------------------------------------------------|---------------------------------------------------|
| 6  | def precision_recall_curve_plot(...)                                     | 3.3장에서 만든 함수를 그대로 재사용합니다.                         |
| 17 | precision_recall_curve_plot(y_test, lr_clf.predict_proba(X_test)[: , 1]) | <b>SMOTE로 학습한 모델의 곡선을 그려 문제의 원인을 시각적으로 진단합니다.</b> |

### ▶ 실행 결과



SMOTE 적용 로지스틱 회귀의 정밀도(점선)-재현율(실선) 곡선

#### ■ 그래프 해석 — 정상적인 모델과 무엇이 다른가

- 임계값이 0.99에 도달할 때까지도 정밀도가 거의 0에 머물립니다. 정상적인 모델이라면 임계값을 높일수록 정밀도가 완만하게 올라가야 합니다.
- 정밀도가 맨 끝에서야 급격히 치솟는 형태입니다. 이는 모델이 거의 모든 샘플에 높은 사기 확률을 부여하고 있다는 뜻입니다.
- 3.3장 타이타닉의 곡선과 비교하면 차이가 뚜렷합니다. 그때는 두 곡선이 완만하게 교차했습니다.

즉 임계값을 아무리 조정해도 이 모델로는 정밀도와 재현율의 균형을 맞출 수 없습니다. SMOTE가 로지스틱 회귀에는 과했다는 결론입니다.

## 5.4 LightGBM 결과

```
lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1, boost_from_average=False)
get_model_train_eval(lgbm_clf, ftr_train=X_train_over, ftr_test=X_test,
                    tgt_train=y_train_over, tgt_test=y_test)
```

#### ▶ 실행 결과

```
오차 행렬
[[85283  12]
 [  22 124]]
정확도: 0.9996, 정밀도: 0.9118, 재현율: 0.8493,   F1: 0.8794, AUC:0.9814
(학습 35.5초)
```

### ■ 결과 해석 — 같은 SMOTE, 전혀 다른 결과

| 모델       | SMOTE | 정밀도           | 재현율           | F1            | AUC           |
|----------|-------|---------------|---------------|---------------|---------------|
| 로지스틱     | 미적용   | 0.8673        | 0.6712        | 0.7568        | 0.9725        |
|          | 적용    | <b>0.0541</b> | 0.9247        | <b>0.1022</b> | 0.9736        |
| LightGBM | 미적용   | 0.9603        | 0.8288        | <b>0.8897</b> | 0.9780        |
|          | 적용    | 0.9118        | <b>0.8493</b> | 0.8794        | <b>0.9814</b> |

- LightGBM은 SMOTE에 훨씬 안정적입니다. 정밀도가 0.9603 → 0.9118로 **소폭 하락**했을 뿐, 로지스틱처럼 붕괴하지 않았습니다.
- 재현율은 0.8288 → 0.8493으로 향상되어 놓치는 사기가 25건 → 22건으로 줄었습니다.
- AUC는 0.9780 → 0.9814로 최고치를 기록했습니다.
- 다만 F1은 0.8897 → 0.8794로 **소폭 하락**했습니다. 정밀도 손실이 재현율 이득보다 약간 컸기 때문입니다.

어느 것이 최선인가? 업무 목표에 달려 있습니다. **사기를 최대한 놓치지 않는 것**이 중요하다면 SMOTE 적용본(재현율 0.8493), **오탐을 줄이는 것**이 중요하다면 미적용본(정밀도 0.9603)을 선택합니다.

## 6. 핵심 요약 및 머신러닝 활용 포인트

### 6.1 전체 실험 결과 종합

| 단계               | 로지스틱 회귀 |               |               | LightGBM      |               |               |
|------------------|---------|---------------|---------------|---------------|---------------|---------------|
|                  | 정밀도     | 재현율           | F1            | 정밀도           | 재현율           | F1            |
| ① 기본 (Time 제거)   | 0.8667  | 0.6149        | 0.7194        | 0.9573        | 0.7568        | 0.8453        |
| ② StandardScaler | 0.8692  | 0.6284        | 0.7294        | 0.9569        | 0.7500        | 0.8409        |
| ③ 로그 변환          | 0.8725  | 0.6014        | 0.7120        | 0.9576        | 0.7635        | 0.8496        |
| ④ + 이상치 제거       | 0.8673  | 0.6712        | <b>0.7568</b> | <b>0.9603</b> | 0.8288        | <b>0.8897</b> |
| ⑤ + SMOTE        | 0.0541  | <b>0.9247</b> | 0.1022        | 0.9118        | <b>0.8493</b> | 0.8794        |

### ■ 최종 권장 모델

LightGBM + 로그 변환 + 이상치 제거가 F1 0.8897로 가장 균형이 좋습니다. 재현율을 더 높여야 한다면 여기에 SMOTE를 추가하거나 임계값을 낮추는 방법을 검토합니다.

### 6.2 핵심 API 요약표

| API                                       | 역할             | 주의 사항                |
|-------------------------------------------|----------------|----------------------|
| <code>train_test_split(stratify=y)</code> | 계층적 분할         | 불균형 데이터에 필수          |
| <code>np.log1p(x)</code>                  | $\log(1+x)$ 변환 | 0을 안전하게 처리. 왜곡 분포 보정 |
| <code>df.insert(0, '이름', 값)</code>        | 지정 위치에 컬럼 삽입   | 단순 대입과 달리 위치 지정 가능   |

|                                             |                 |                                                       |
|---------------------------------------------|-----------------|-------------------------------------------------------|
| <code>np.percentile(arr, 25)</code>         | 분위수 계산          | IQR = Q3 - Q1                                         |
| <code>df.corr()</code>                      | 상관 행렬           | 레이블과의 상관으로 <b>중요 피처 탐색</b>                            |
| <code>sns.heatmap(corr, cmap='RdBu')</code> | 상관 시각화          | 발산형 컬러맵 사용                                            |
| <code>SMOTE().fit_resample(X, y)</code>     | 오버 샘플링          | <b>학습 데이터에만 적용</b><br><code>fit_transform</code> 이 아님 |
| <code>boost_from_average=False</code>       | LightGBM 시작점 설정 | <b>극심한 불균형에서 필수</b>                                   |

## 6.3 머신러닝 활용 포인트

- **전처리 함수를 교체 가능하게 설계** : `get_preprocessed_df()` 를 재정의하는 구조 덕분에 5가지 전처리를 **같은 코드로** 비교할 수 있었습니다. 실험 설계의 좋은 본보기입니다.
- **전처리 효과는 알고리즘마다 다르다** : 로그 변환이 로지스틱에는 해로웠지만 LightGBM에는 이로웠습니다. **전처리와 모델은 짝** 으로 평가해야 합니다.
- **이상치 제거의 가성비** : 28만 건에서 **4건**을 지웠을 뿐인데 재현율이 6~7%p 올랐습니다. **모델 튜닝보다 데이터 품질**이 효과적일 수 있습니다.
- **SMOTE는 만능이 아니다** : 재현율은 확실히 올리지만 **정밀도를 대가로** 지불합니다. 특히 **선형 모델에는 치명적**일 수 있습니다. 반드시 적용 전후를 비교하세요.
- **이상치는 레이블과 상관 높은 피처에서** : 히트맵으로 V14를 고른 것처럼, **근거를 가지고** 대상을 선택해야 합니다.
- **업무 맥락이 최종 판단 기준** : F1이 가장 높은 모델이 항상 정답은 아닙니다. **사기 1건을 놓치는 비용과 정상 거래를 차단하는 비용**을 비교해 결정해야 합니다.

### ⚠️ 안정화 버전 참고

- **imbalanced-learn**은 별도 설치가 필요합니다: `pip install imbalanced-learn` . 임포트는 `imblearn` 입니다.
- **LightGBM 4.x**: 학습 로그를 끄려면 생성자에 `verbose=-1` 을 지정하세요.
- `LogisticRegression()` 의 기본 `max_iter=100` 은 이 데이터에서 **수렴 경고**가 발생할 수 있습니다. `max_iter=1000` 으로 늘리거나 스케일링을 적용하세요. 본 문서의 실행 결과는 `max_iter=1000` 기준입니다.
- `boost_from_average` 의 기본값은 `True` 입니다. **불균형이 극심한 데이터에서는 반드시 `False` 로** 설정하세요. 그렇지 않으면 재현율이 크게 떨어집니다.
- SMOTE는 `Pipeline` 에 넣을 때 사이킷런의 `Pipeline` 대신 `imblearn.pipeline.Pipeline` 을 써야 교차 검증 시 **폴드마다 올바르게** 적용됩니다.